

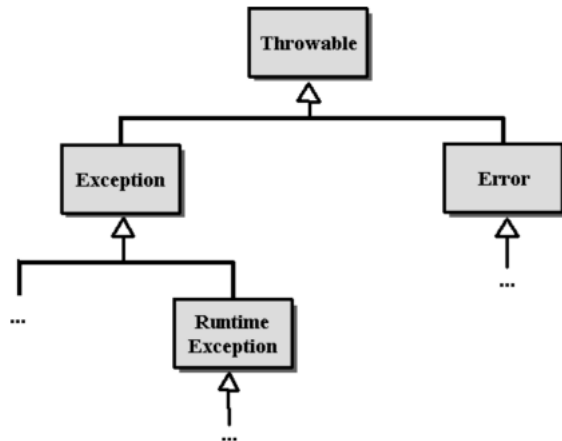
A. Exception Handling

1. Fehler in Java

+ **Compiler Fehler:** syntaktische Fehler werden beim Kompilieren erkannt.

+ **Laufzeitfehler:**

- Fehler sollte nicht behandelt werden.
- Ausnahmen (Exceptions)
 - Exception muss behandelt werden
 - RuntimeException kann behandelt werden.



2. Grundprinzip der Ausnahmebehandlung:

+ Laufzeitfehler oder explizite Anweisung löst Ausnahme aus.

+ 2 Möglichkeiten der Fehlerbehandlung:

- Direkte Fehlerbehandlung im auslösenden Programmteil
- Weitergabe der Ausnahme an die aufrufenden Methode: Bei Weitergabe liegt die Entscheidung beim Empfänger:
 - Er kann die Ausnahme behandeln.
 - Er kann die Ausnahme an seinen Aufrufer weitergeben.

+ wird die Ausnahme nicht behandelt, führt sie zur Ausgabe einer Fehlermeldung und zum Programmabbruch (Laufzeitfehler).

3. Ausnahme behandeln:

+ try: markiert den Überwachungsbereich – ausgelöste Ausnahmen beenden die Ausführung des Überwachungsbereiches umgehend.

+ catch: fängt mögliche Ausnahmen aus dem Überwachungsbereich auf.

+ finally: wird unabhängig vom Auftreten von Ausnahmen zum Abschluss des try-catch-Blocks ausgeführt

4. Try-with-Resource

+ vermeiden unschöner Schachtelung von try-Blöcken (aber: andere Reihenfolge im Vergleich zur finally Ausführung)

+ Voraussetzung: Resource implementiert Closable Interface.

+ automatisches schließen der Ressourcen nach try-Block

- Exception beim schließen werden innerhalb der Catch-Blöcke mit abgefangen.

5. Wichtige Methode der Klasse Throwable

`public String getMessage()`

- liefert den Fehlertext zurück

```
Der Tank ist nach 1100 Kilometern leer.
```

`public String toString()`

- liefert die Objektbeschreibung und den Fehlertext zurück

```
prog2.demos.exceptions.TankLeer: Der Tank ist nach 1100 Kilometern leer.
```

`public void printStackTrace()`

- liefert die Objektbeschreibung, den Fehlertext sowie die Weitergabehierarchie bis zur genauen Auslösestelle zurück

```
prog2.demos.exceptions.TankLeer: Der Tank ist nach 1100 Kilometern leer.  
at prog2.demos.exceptions.Auto.fahren(Auto.java:21)
```

6. Checked vs. Unchecked Exceptions

Checked

- müssen verarbeitet werden
 - abfangen mit `try / catch`
 - weiterleiten mit `throws`
- werden explizit ausgelöst `throw`

Unchecked

- treten zur Laufzeit auf (`RuntimeException`)
- werden automatisch an den Aufrufer weitergegeben
- können abgefangen werden `try / catch`
- oftmals logische Programmfehler
 - `Division by Zero`
 - `NullPointerException`
 - `IndexOutOfBoundsException`

7. Ausnahmen in JUnit-Tests

spezielle Assertion

- Rückgabe des Ausnahme Objektes
- schlägt fehl, wenn keine oder eine andere Ausnahme zurück geworfen wird

```
Assertions.assertThrows(<Erwartete Ausnahme Klasse>. <Executable Interface>[, <Message>]);
```

Assertion für den gegenläufigen Fall

- schlägt fehl, wenn eine Ausnahme geworfen wird

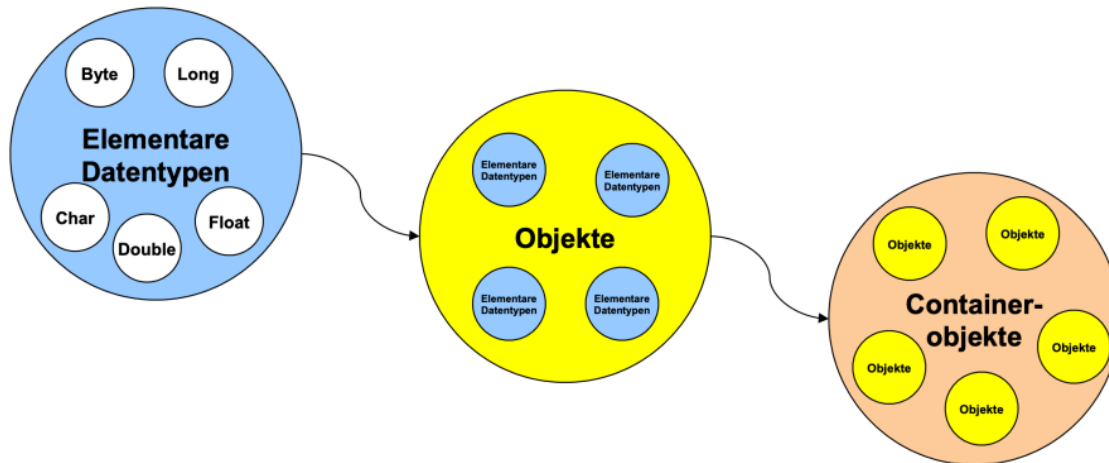
```
Assertions.assertDoesNotThrow(<Executable Interface> [, <Message>]);
```

(!) JUnit4: spezielles Attribut in `@Test` Annotation

```
@Test(expected = <Erwartete Ausnahme Klasse>)
```

B. Collection Framework

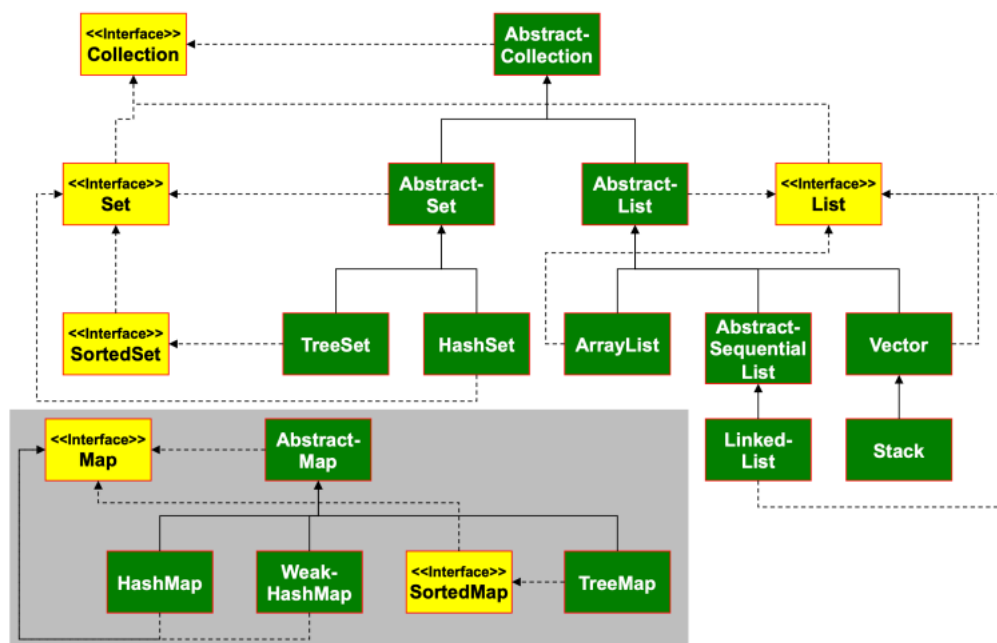
1. Datenstrukturen und -container:



+ Das Collection Framework bietet generische Container:

- Können verschiedenste Objekte enthalten
- Können beliebig viele Objekte aufnehmen
- Können auf bestimmte Objekte typisiert werden

2. Überblick über das Collection Framework



3. Die 3 Arten von Containern

+ Listen (List)

- Zugriff sequentiell oder wahlfrei
- Duplikate erlaubt
- Reihenfolge des Einfügens bleibt erhalten

+ Mengen (Set)

- Zugriff erfolgt über Iteratoren
- Keine Duplikate
- Reihenfolge des Einfügens bleibt nicht erhalten

+ Schlüssel-Werte-Paare (Map)

- Zusammengehörige Objektpaare
- Schlüssel sind immer eindeutig
- Zugriff über Schlüssel

4. Das Interface List

- + befindet sich in Package java.util
- + Zugriff auf die Container erfolgt sequentiell (über Iteratoren) oder wahlfrei (über Index)
- + Index beginnt mit 0 und endet bei n Elementen bei n-1
- + Größe der Liste wird dynamisch beim Einfügen oder Löschen von Elementen angepasst.
- + Duplikate sind erlaubt.
- + die Reihenfolge, in der Elemente eingefügt werden, bleibt erhalten.
- + meist genutzte Implementierung: ArrayList&Vector
- Intern als Array realisiert
- Hauptunterschied zwischen ArrayList und Vector: Zugriffsmethoden auf Vector sind synchronisiert (wichtig bei Threads)

5. Wesentliche Methoden im Umgang mit Listen

- `add(int i, Object o)` oder `add(Object o)` fügt neue Objekte in die Liste ein
- `set(int i, Object o)` überschreibt das Objekt an der Stelle i mit dem Objekt o
- `get(int i)` liefert das Objekt an der Stelle i zurück
- `contains(Object o)` überprüft, ob das Objekt o in der Liste enthalten ist
- `indexOf(Object o)` liefert den Index zurück, an der das Objekt o in der Liste abgelegt ist (-1, wenn das Objekt nicht enthalten ist)
- `remove(int i)` oder `remove(Object o)` löscht das Objekt aus der Liste
- `clear()` initialisiert die Liste
- `size()` liefert die Länge der Liste zurück

6. Merkmale von Iteratoren

- + einheitlicher Standard zum Durchlaufen von Datencontainern
- + Container wird sequentiell durchlaufen
- + es können keine Elemente übersprungen werden.
- + der Container kann sowohl vorwärts als auch rückwärts durchlaufen werden
- + bei Änderung des Containerinhalts muss der Iterator neu erzeugt werden.

7. Wichtige Iterator-Methoden

- `hasNext()` überprüft, ob das aktuelle Element im Container noch einen Nachfolger hat
- `next()` greift auf das nächste Element des Containers zu
- `remove()` löscht das Element aus dem Container, welches zuletzt vom Iterator gelesen wurde

8. Die Klasse TreeSet

- + befindet sich im Package java.util
- + Zugriff auf die Container erfolgt über Iteratoren
- + Größe der Liste wird dynamisch beim Einfügen oder Löschen von Elementen angepasst.
- + Duplikate sind nicht erlaubt (Vergleich über die equals-Methode)
- + Die Reihenfolge, in der Elemente eingefügt werden, bleibt nicht erhalten.
- + implementiert SortedSet-Interface; sortiert nach einer vorgegebenen Ordnung
- Natürliche Ordnung durch das Comparable-Interface
- Beliebige/externe Sortierung durch ein Objekt entsprechend dem Comparator-Interface.

9. Das Interface Comparable

- + sortiert Elemente beim Einfügen in Sets oder Maps (welche auf SortedSet basieren)
- + soll die natürliche Ordnung der Objekte widerspiegeln
- + Voraussetzung für die Datencontainer vom typ SortedSet oder SortedMap (sofern keine externe Sortierung über eine Comparator Implementierung gegeben ist)

- beinhaltet genau eine Methode: `public int compareTo(Object o)`
- Bedeutung der Rückgabewert -->
`aufgerufenesObjekt.compareTo(übergebenesObjekt)`
 - Wert < 0: das aufgerufene Objekt ist kleiner als das übergebene Objekt
 - Wert = 0: das aufgerufene Objekt ist gleich dem übergebenen Objekt
 - Wert > 0: das aufgerufene Objekt ist größer als das übergebene Objekt
- Beispiel:

```
"a".compareTo("c"); // --> ergibt -2
"c".compareTo("c"); // --> ergibt 0
"c".compareTo("a"); // --> ergibt 2
```

10. Das Interface Comparator

- + sortiert Elemente beim Einfügen in Sets oder Maps.
- + Sortierung erfolgt nach einer beliebigen Sortierreihenfolge und übersteuert die natürliche Ordnung.
- + Comparator sollten in eigener Klasse implementiert werden (auch innere Klassen oder Lamdafunktionen)
- + Zur Verwendung des Comparators wird ein Objekt der implementierende Klasse dem Konstruktor des Sets oder der Map übergeben.

- beinhaltet genau eine Methode: `public int compare(Object o1, Object o2)`
- Bedeutung der Rückgabewerte
 - Wert < 0: o1 liegt vor o2
 - Wert = 0: o1 und o2 sind gleich
 - Wert > 0: o1 liegt hinter o2

11. Sortieren von Listen

- Listen (Vector, ArrayList, ...) sind normalerweise unsortiert
- die Klasse Collections bietet eine überladene Sortiermethode zum Sortieren von List-Objekten an
- folgende Sortiermöglichkeiten werden angeboten
 - `static void sort(List liste)`
 - sortiert die Liste nach der natürlichen Ordnung
 - dazu müssen die Klassen das Interface Comparable implementieren, deren Instanzen in der Liste gespeichert sind
 - `static void sort(List liste, Comparator c)`
 - übersteuert die natürliche Ordnung und sortiert die Objekte der Liste über den entsprechenden Comparator c

12. Vergleich von Objekten

- Vergleich mit dem ==-Operator prüft, ob es sich um die identische Speicherreferenz handelt
- inhaltliche Vergleiche erfolgen über die equals() -Methode (equals() -Methode der Klasse Object entspricht dem ==-Operator)
- der equals-Contract aus der Dokumentation zur Klasse Object
 - reflexiv: jedes Objekt liefert beim Vergleich mit sich selbst true
 - symmetrisch: x verglichen mit y liefert das gleiche Ergebnis, wie der Vergleich von y mit x
 - transitiv: wenn x gleich y und y gleich z ist, dann ist auch x gleich z
 - konsistent: solange sich zwei Objekte nicht verändern, liefert der Vergleich der beiden Objekte immer das gleiche Ergebnis
 - Objekte müssen von null verschieden sein

13. Das Überschreiben der equals()-Methode

direkte Sub-Klasse von Object	indirekte Sub-Klasse von Object
• Alias-Check mit dem ==-Operator • Test auf null • Typverträglichkeit überprüft, ob es sich um Instanzen der gleichen Klasse handelt • Feld-Vergleich überprüft die inhaltliche Gleichheit der Attribute	• Alias-Check mit dem ==-Operator • Delegation an die Oberklasse ermöglicht die Prüfung der Gleichheit der von der Oberklasse geerbten Anteile • Feld-Vergleich überprüft die inhaltliche Gleichheit der Attribute der Sub-Klasse

14. Zusammenhang hashCode() und equals()

- Verwendung für die Verwaltung der Einträge in hash-basierten Datencontainern (HashSet, HashMap, ...)
- korrekte Verwaltung der Einträge basiert auf folgender Bedingung (hashCode-Contract)
 - wenn o1.equals(o2) den Wert true liefert,
 - dann muss o1.hashCode() den gleichen Wert ergeben, wie o2.hashCode()
- sobald die equals() -Methode überschrieben wird, muss auch die hashCode() -Methode überschrieben werden, so dass o.g. Bedingung erfüllt wird
- Vorschlag zur Implementierung
 - Verwendung der Attribute, die bei der Implementierung der

equals() -Methode verwendet werden

- Ermittlung der Hash-Codes der ausgewählten Attribute einer Klasse
- Addition oder bitweise Verknüpfung mit exklusivem Oder der einzelnen Hash-Codes

15. hashCode() – Alternative Implementierung

Typ	Zugeordneter Integer Wert
Boolean	(field ? 0 : 1)
byte, char, short, int	(int) field
long	(int)(field>>>32) ^ (int)(field & 0xFFFFFFFF)
float	((x==0.0F) ? 0 : Float.floatToIntBits(field))
double	((x==0.0) ? 0L : Double.doubleToLongBits(field)) [anschliessende Behandlung wie bei long]
Referenz	((field==null) ? 0 : field.hashCode())

Was hat das mit Comparable zu tun?

- `compareTo()` sortiert Objekte nach einer "natürlichen" Ordnung
 - Rückgabe Wert 0: die Objekte sind gleich
 - damit sollte der Rückgabewert 0 für zwei Objekte einem Rückgabewert von true beim Vergleich mit `equals()` entsprechenden ([Comparable-Contract](#))

`equals()` und `compareTo()` sollten sich konsistent verhalten

- Zusammengefasst: `equals()`, `hashCode()` und `compareTo()` **sollten** für ein Objekt immer auf den gleichen Attributen basieren

16.

17. Das Interface Map

- + befindet sich im Package `java.util`
- + ist kein Sub-Interface von `Collection`
- + es werden immer Schlüssel-Werte-Paare eingefügt.
- + jeder Schlüssel ist eindeutig
- + wird mit dem gleichen Schlüssel ein weiterer Wert eingefügt, so wird der erste Wert überschrieben.
- + Zugriff auf die Werte-Objekte erfolgt über die Schlüssel.

2 wesentliche Vertreter

- `TreeMap`: Einträge werden nach Schlüssel sortiert -> Schlüssel-Klasse muss das Interface `Comparable` implementieren.
- `HashMap`: auf Basis der `hashCode()`-Methode der Schlüsselklasse wird eine interne Position (Bucket) berechnet, an der Schlüssel-Werte-Paar in die Map aufgenommen wird.

18. Wesentliche Methoden im Umgang mit Maps

- `keySet()` liefert ein Set der Schlüssel einer Map ohne Duplikate zurück
- `values()` liefert eine Collection der Werte einer Map zurück (Duplikate erlaubt)
- `put(Object k, Object v)` nimmt ein Schlüssel-Werte-Paar in die Map auf
- `get(Object k)` liefert den Wert zum Schlüssel-Objekt k zurück | `containsKey(Object k)` liefert true zurück, wenn zu dem

Schlüssel k ein Eintrag in der Map enthalten ist

- `containsValue(Object v)` liefert true zurück, wenn zu dem Wert v ein Eintrag in der Map enthalten ist
- `remove(Object k)` löscht den Eintrag zum Schlüssel k aus der Map
- `size()` liefert die Länge der Map zurück
- `clear()` initialisiert die Map

19. Umgang mit Wrapper-Klassen

- + statt elementarer Datentypen werden Objekte erwartet (z.B. in Datencontainern)
- + um elementare Datentypen in Objekten zu kapseln, gibt es die Wrapper-Klassen
- Stellen Methoden zur Ein- und Ausgabe sowie zur Manipulation zur Verfügung
- Stellen Methoden zur Umwandlung von Datentypen zur Verfügung
 - + Wrapper-Klassen existieren für folgende Datentypen: `boolean`, `byte`, `char`, `double`, `float`, `int`, `long`, `short`.

+ Auto-Boxing/ Auto-Unboxing

- Java erstellt automatisch ein Objekt der passenden Wrapper-Klasse wenn ein Objekt erwartet, aber ein einfacher Datentyp bereitgestellt wird (Auto-Boxing)
- Umgekehrt wird der Wert als einfacher Datentyp bereitgestellt, wenn ein Objekt der Wrapper-Klasse zurückgegeben wird (Auto-Unboxing)

C. Swing

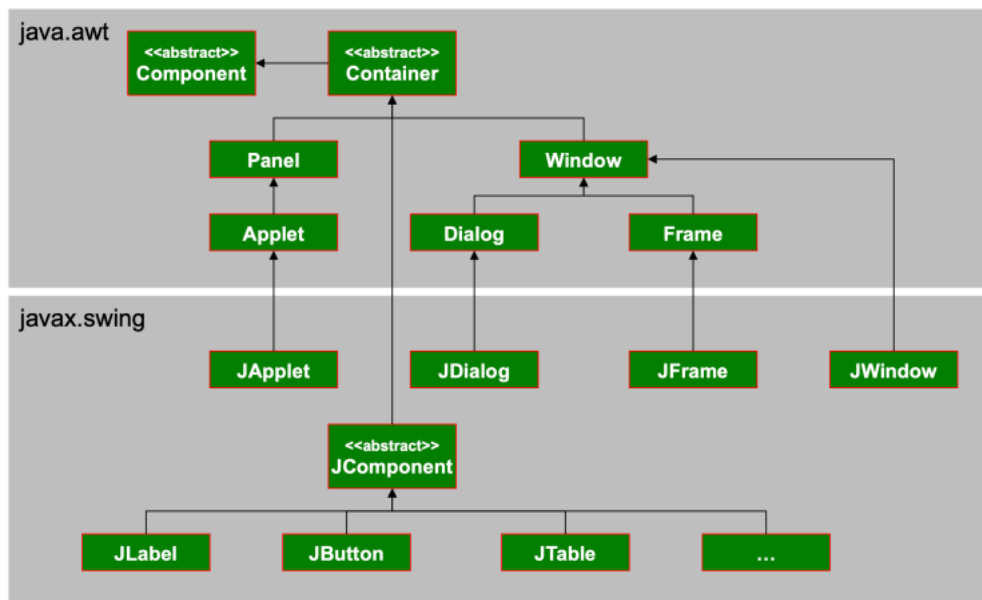
1. Abgrenzung von AWT und Swing

AWT (Abstract Window Toolkit) arbeitet mit „Heavyweight components“

- Verwendung von plattformspezifischen Implementierungen der AWT-Klassen (nicht in Java implementiert !)
- AWT-Komponenten besitzen einen Partner auf Betriebssystemseite (Peer), der Darstellung und Funktionalität steuert
- Vorteil: sehr schnell, da die Peer-Klassen im Code der Ausführungsplattform geschrieben sind

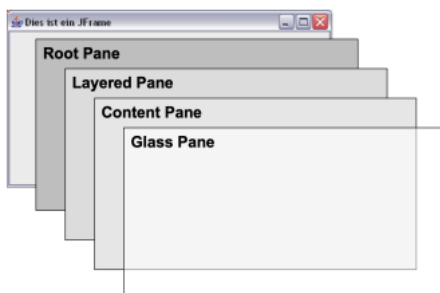
Swing arbeitet mit „Lightweight components“

- es werden nur sehr wenige plattformspezifische GUI-Ressourcen verwendet
- lightweight components besitzen keinen Peer auf Betriebssystemseite
- Swing besitzt zahlreiche zusätzliche GUI-Komponenten
- Vorteil: „bessere“ Plattformunabhängigkeit
- Nachteil: im Vergleich zu AWT eher langsam



2. Aufbau eines Swing-Fensters mit JFrame

- + Hauptkomponente eines JFrames ist die `RootPane`
- + darunter folgt eine Hierarchie (Panels)
- + neue Komponenten werden der `ContentPane` zugeordnet und nicht dem `JFrame`



3. Wichtige Methoden für JFrames

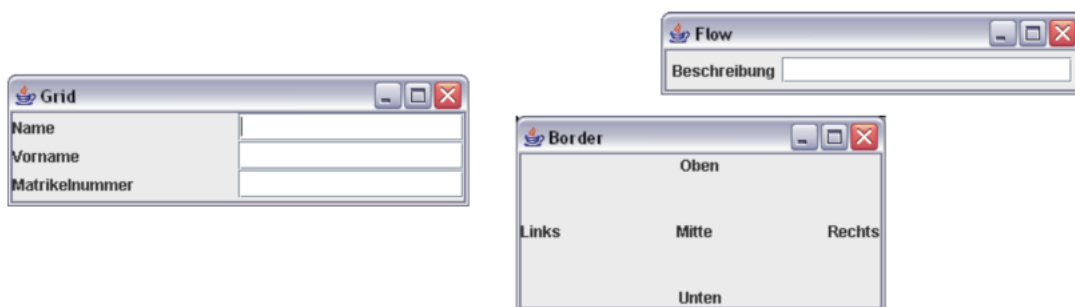
- überladener Konstruktor, u.a. zum Setzen des Titels
- `setDefaultCloseOperation(int i)` legt fest, was beim Schließen

des Fensters passiert

- Konstanten, die o.g. Methode übergeben werden können
 - `WindowConstants.DO_NOTHING_ON_CLOSE` löst lediglich das Close-Event aus
 - `WindowConstants.HIDE_ON_CLOSE` versteckt das Fenster
 - `WindowConstants.DISPOSE_ON_CLOSE` zerstört den Frame
 - `WindowConstants.EXIT_ON_CLOSE` beendet die Applikation
- Getter- und Setter-Methoden für die Panels eines JFrames, z.B. `getContentPane()`
- Methoden aus der Klasse `java.awt.Window`
 - `setBounds(int x, int y, int width, int height)`
 - `pack()` passt die Fenstergröße an den Content an
- `setVisible(boolean b)` aus der Klasse `java.awt.Component`

4. Layouts im Rahmen von Swing

- Anordnung der Elemente eines Containers nach bestimmten Verfahren über Layout-Manager
- wesentliche Layout-Manager
 - `FlowLayout` ordnet seine Elemente von links nach rechts
 - `BorderLayout` ermöglicht eine Anordnung in 5 verschiedenen Bereichen (NORTH, EAST, SOUTH, WEST und CENTER)
 - `GridLayout` ermöglicht die Anordnung der Komponenten in Zeilen und Spalten von links nach rechts und von oben nach unten
- mit der Methode `setLayout(LayoutManager l)` wird für ein JFrame der Layout-Manager gesetzt



5. Der Container JPanel

- JPanel ist eine weitere Container-Form
- ordnet mehrere Elemente unter der Kontrolle eines Layoutmanagers an
- Layoutmanager und Komponenten werden direkt dem Panel zugewiesen
- bereits dem Konstruktor wird der Layoutmanager mitgegeben | über die `add()`-Methode werden die Komponenten dem Panel zugeordnet

6. Panels mit Rahmen hervorheben

- Rahmen sind über Klassen realisiert, die das Interface `Border` implementieren
- Rahmen sollten nicht direkt über die Konstruktoren der Rahmen-Klassen sondern über die Klassenmethoden der `BorderFactory` erzeugt werden
- jeder Swing-Komponente kann mit der Methode `setBorder(Border b)` ein Rahmen zugewiesen werden
- einige Standardrahmen sind in Swing bereits implementiert

7. Verschiedene Rahmen

Klasse	Rahmenart
AbstractBorder	eine abstrakte Klasse, die die Schnittstelle minimal implementiert
BevelBorder	ein 3D-Rahmen, der eingelassen sein kann
CompoundBorder	ein Rahmen, der andere Rahmen aufnehmen kann
EmptyBorder	Rahmen, dem freier Platz zugewiesen werden kann
EtchedBorder	noch deutlicher markierter Rahmen
LineBorder	Rahmen in einer einfachen Farbe in gewünschter Dicke
MatteBorder	Rahmen, bestehend aus Kacheln von Icons
SoftBevelBorder	ein 3D-Rahmen mit besonderen Ecken
TitledBorder	Rahmen mit String in einer gewünschten Ecke

8. Swing UI Komponenten

8.1 Beschriftungen und Grafikanzeige mit JLabel

- ermöglicht einfache Anzeige von Texten oder Grafiken
- zu einem Text kann zusätzlich ein Icon angezeigt werden
- bietet die Möglichkeit, HTML-Tags darzustellen
- häufiger Einsatz zur Beschriftung anderer Dialogkomponenten

8.2 Unterschiedliche Arten von Textfeldern

- einfache Textfelder der Klasse [JTextField](#)
 - überladener Konstruktor, um das Feld mit einem String vorzugeben und/oder die Breite anzugeben
 - Angabe der Schriftart über die Methode `setFont()`
 - Auslesen des Inhalts über die Methode `getText()`
- spezielle Felder für Passwörter der Klasse [JPasswordField](#)
 - Konstruktoren analog der Klasse [JTextField](#)
 - Auslesen des Inhalts über die Methode `getPassword()`
 - zwei boolesche Methoden `cut()` und `copy()`, die überprüfen, ob Werte mit `cut` (STRG+X) oder `copy` (STRG+C) aus dem Feld ausgelesen werden dürfen
- mehrzeilige Textfelder der Klasse [JTextArea](#)
 - Konstruktoren analog der Klasse [JTextField](#) - Unterschied: es muss neben der Breite auch die Höhe des Feldes angegeben werden
 - Auslesen und ändern der Schriftart analog der Klasse [JTextField](#)
 - Zeilenumbrüche werden bei `getText()` berücksichtigt

8.3 Validierende Textfelder als spezielle Form

- realisiert durch die Klasse [JFormattedTextField](#)
- dem Konstruktor der Klasse wird das Format mitgegeben
- mehrere Klassen stehen für die Maskierung zur Verfügung
 - alle Objekte der Sub-Klassen der Klasse `Format` (z.B.

`SimpleDateFormat`, `DecimalFormat`, etc.)

- z.B. bei Drücken der Enter-Taste wird die Eingabe überprüft und ein mögliches `ActionEvent` ausgelöst
- Objekte der Klasse [MaskFormatter](#) erlauben nur bestimmte Zeichen bei der Eingabe

Platzhalter	Beschreibung
#	nur Ziffern sind erlaubt
,	Escape-Zeichen als Prefix vor einem Platzhalter
U	erlaubt nur Buchstaben, Kleinbuchstaben werden zu Großbuchstaben konvertiert
L	erlaubt nur Buchstaben, Großbuchstaben werden zu Kleinbuchstaben konvertiert
A	nur Ziffern oder Buchstaben sind erlaubt
?	nur Buchstaben sind erlaubt
*	alle Zeichen sind erlaubt
H	nur Zeichen zur Hexadezimaldarstellung sind erlaubt (0-9 und A-F)

8.4 Drop-Down-Listen über JComboBox

- eine bestimmte Wertemenge wird zur Auswahl bereit gestellt
- dem Konstruktor der Klasse JComboBox wird die Wertemenge als ein Array von Objekten der Klasse Object übergeben
- wesentliche Methoden der Klasse JComboBox
 - `getSelectedItem()` liefert den Wert des ausgewählten Elements zurück (entspricht der Methode `getText()` bei `TextField`)
 - `setSelectedItem(Object o)` belegt das Feld mit dem Wert `o` vor, sofern dieser in dem Array der Wertemenge vorhanden ist
 - `setEditable(boolean b)` bestimmt, ob auch Werte außerhalb der Wertemenge erlaubt sind
 - `b = true` -> freie Eingabe erlaubt
 - `b = false` -> freie Eingabe nicht erlaubt

8.5 Weitere Klassen aus dem Package Swing

- **JTable**
 - dient der Erstellung zweidimensionaler Tabellen
- **JTree**
 - ermöglicht die Darstellung von Bäumen ähnlich dem Windows Explorer bestehend aus Knoten und Blättern
- **JToolBar**
 - dient der Erstellung von Symbolleisten analog den Microsoft Office-Produkten
- **JColorChooser**
 - dient der Erstellung eines Auswahldialogs zur Farbeinstellung
- **JFileChooser**
 - dient der Erstellung eines Dialogs zur Auswahl einer Datei im FileSystem
- ...

9. EventListener

9.1 Das Event Konzept

- UI Komponenten erzeugen Events z.B. klick auf einen Button (**ActionEvent**), ändern einer Auswahl (**ItemEvent**), wechseln eines Fensters (**FocusEvent**) ...
- Jeder Event-Typ hat ein Listener Interface um auf das Event reagieren zu können
 - **ActionEvent** --> **ActionListener**
 - **ItemEvent** --> **ItemListener**
 - **FocusEvent** --> **FocusListener**
 - ...
- Komponenten, welche Events erzeugen, können (mehrere) Implementierungen des jeweiligen Interfaces registrieren (z.B. `addActionListener(ActionListener a)`)
- Tritt ein Event auf, wird die jeweilige Methode der registrierten Interface-Implementierung aufgerufen



9.2 Die Aufgaben des ItemListener

- der `ItemListener` ist als Interface implementiert
- das Interface gibt die abstrakte Methode `itemStateChanged(ItemEvent e)` vor
- das Interface wird von Objekten implementiert, die an einem Auswahlereignis interessiert sind
- Auswahlereignisse können von Objekten folgender Klassen ausgelöst werden: `JComboBox`, `JCheckBox`, `JList` oder `JCheckBoxMenuItem`
- die Zuordnung zu einem `ItemListener` erfolgt über die jeweiligen Objekt-Methoden `addItemListener()` oder `removeItemListener()`
- wird ein Eintrag bei o.g. Objekten ausgewählt, wird implizit die Methode `itemStateChanged(ItemEvent e)` bei allen bei dem Objekt registrierten `ItemListener`en ausgeführt
- Beispiel: beim Setzen des Hakens wird ein zusätzliches Feld eingeblendet

9.3 Interaktion über Drucktasten mit `JButton`

- überladener Konstruktor, der es ermöglicht Text und oder Grafik in Form eines Icon auf dem Button zu positionieren
- mit der Methode `setText(String s)` kann der Text nachträglich verändert werden
- wichtigste Methoden `addActionListener()` und `removeActionListener()`
- der `ActionListener` ist der Beobachter des Knopfes
- ohne `ActionListener` kann dem Button keine Funktionalität zugewiesen werden
- sobald der Button gedrückt wird, wird ein `ActionEvent` ausgelöst, welches vom Beobachter abgefangen und ausgewertet wird

9.4 Aufgaben des `ActionListener`

- der `ActionListener` ist als Interface implementiert
- das Interface gibt die abstrakte Methode `actionPerformed(ActionEvent e)` vor
- diese Methode wird implizit ausgeführt, sobald ein „abgehörtes“ Objekt ein `ActionEvent` auslöst
- die Klasse `ActionEvent` besteht aus drei Methoden
 - `getActionCommand()` liefert den String, der mit der Aktion verbunden ist (bei `JButton` die Beschriftung des Buttons)
 - `getModifiers()` liefert einen Integer-Wert zurück, welche Funktionstaste bei dem Ereignis gedrückt wurde (Shift, Alt, etc.)
 - `paramString()` liefert einen Erkennungs-String, der mit „ACTION_PERFORMED“ oder „unknown type“ beginnt

9.5 Kontrollfelder mit `JCheckBox`

- Kontrollfelder kennen zwei Zustände: selektiert (`true`) und nicht selektiert (`false`)
- überladener Konstruktor, der es ermöglicht Text, Initialwert (`true` oder `false`) und Icon mitzugeben
- Kontrollfelder werden normalerweise als Kästchen mit einem Häkchen für den selektierten Zustand dargestellt
- der Zustand kann über die Methode `setSelected(boolean b)` geändert werden
- der Zustand kann allerdings nicht direkt über eine Getter-Methode ausgelesen werden
- bei der Änderung des Zustands durch den Anwender wird ein `ItemEvent` ausgelöst und an alle registrierten `ItemListener` weitergeleitet
- im `ItemListener` kann der Zustand des Kontrollfeldes ausgewertet und weiter verarbeitet werden

9.6 Optionsfelder mit `JRadioButton` & `ButtonGroup`

- Optionsfelder bieten mehrere Auswahlmöglichkeiten an, wobei nur eine Option ausgewählt werden kann
- dazu werden Optionsfelder in einem Objekt der Klasse `ButtonGroup` zu einer Optionsfeldgruppe zusammengefasst
 - mit der Objektmethode `add(AbstractButton b)` der Klasse `ButtonGroup` wird ein Optionsfeld der Gruppe hinzugefügt
 - mit der Objektmethode `remove(AbstractButton b)` der Klasse `ButtonGroup` wird ein Optionsfeld aus der Gruppe entfernt
- überladener Konstruktor der Klasse `JRadioButton` analog der Klasse `JCheckBox`
- Optionsfelder werden normalerweise als Kreis mit einem schwarzen Punkt für den selektierten Zustand dargestellt
- bei der Änderung des Zustands eines Optionsfeldes wird ein `ActionEvent` ausgelöst und an alle registrierten `ActionListener` weitergeleitet
- im `ActionListener` kann die Auswertung der Optionsfelder erfolgen

10. Weitere Komponenten

10.1 Erstellen von Menüs mit Swing-Komponenten

- **JMenuBar** ist der Container für die einzelnen Menüs
 - mit der `add(JMenu m)` Methode wird dem Container ein Menü hinzugefügt
- Objekte der Klasse **JMenu** stellen die einzelnen Menüs dar und sind Container für konkrete Menüeinträge
 - mit der `add(JMenuItem i)` Methode wird einem Menü ein konkreter Menüeintrag zugeordnet
- Objekte der Klasse **JMenuItem** repräsentieren Menüeinträge
- mit der Methode `setJMenuBar(JMenuBar m)` wird einem Fenster eine Menüleiste zugeordnet
- um auf die Auswahl eines Menüeintrags zu reagieren, müssen die Menüeinträge einem `ActionListener` zugeordnet werden

10.2 Tooltips

- Tooltips sind kleinere Hilfetexte, die beim längeren Verweilen auf einem GUI-Objekt in einem kleinen PopUp-Fenster angezeigt werden
- ToolTips werden nicht direkt über den Konstruktor der Klasse `JToolTip` erzeugt, sondern über die Methode `setToolTipText(String s)` des GUI-Objektes
 - der String `s` kann als einfacher Text übergeben werden
 - der String `s` kann im HTML-Format übergeben werden

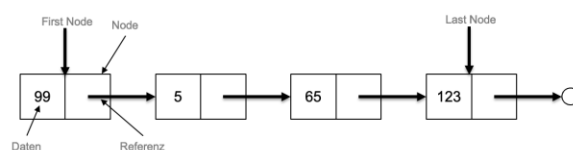
D. Datenstrukturen

1. Listen

1.1 Verkettete Liste (Linked List)

Methoden

- `add()` / `addFirst()` / `addLast()` / `addAfter()`
- `remove(by object / by search attributes)`
- `find(by search attributes)`
- `contains(by object)`
- `getFirst()` / `getLast()`
- ...

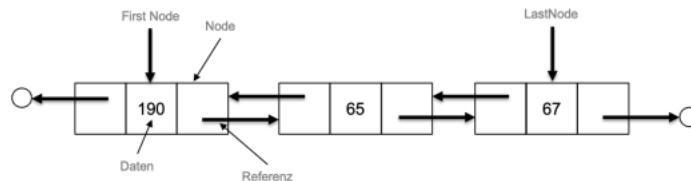


1.2 Doppelt Verkettete Liste (Double Linked List)

Methoden

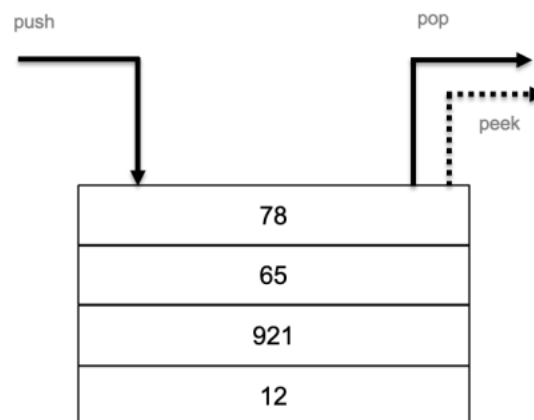
- add() / addFirst() / addLast() / addAfter() / addBefore()
- remove(by object / by search attributes)
- find(by search attributes)
- contains(by object)
- getFirst() / getLast()
- ...

Parallel Suche: $O(n/2) \rightarrow O(n)$



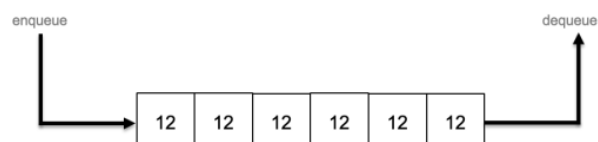
2. Stapel (Stack)

- Eigenschaften
 - Ein-/Ausgabe Strategie: LIFO (Last In - First Out)
 - Neue Elemente werden oben auf den Stapel gelegt, das nächste Element (neueste auf dem Stapel) wird oben vom Stapel entnommen
- Methoden
 - push()
 - pop()
 - peek()



3. Warteschlange (Queue)

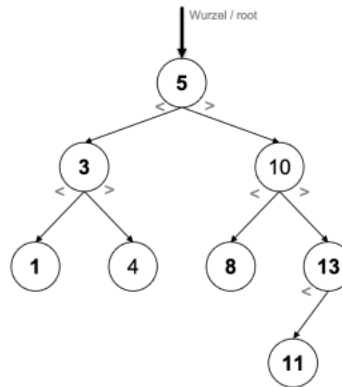
- Eigenschaften
 - Ein-/Ausgabe Strategie: FIFO (First In - First Out)
 - Neue Elemente werden in die Schlange eingereiht, das älteste Element (am längsten in der Schlange) wird als nächstes entnommen
- Methoden
 - enqueue()
 - dequeue()



4. Binär-Baum (Binary tree)

Methoden

- insert(object)
- [find(by search attribute)] (Nicht für generische Container)
- contains(object)
- remove(object)
- ...



5. Iteration vs. Rekursion

- Sowohl Iteration als auch Rekursion werden für sich wiederholende Anweisungen in Algorithmen verwendet
- Iteration nutzt Schleifen zur Aneinanderreihung
- Rekursion nutzt Methoden/Funktionen zur Schachtelung
- Beide Arten benötigen definierte Abbruchkriterien

Eigenschaften von rekursiver Programmierung

- belegt mit jeder neu geschachtelten Ebene extra Speicher auf dem Stack (Arbeitsspeicherbereich)
- jeder (Methoden-/Funktions-)Aufruf für eine neue Ebene braucht extra Rechenleistung (Zeit / Performance)
- -> intensiver im Speicher- und Rechenleistungsverbrauch
- Quellcode bleibt (im speziellen in komplexen Fällen) übersichtlicher

6. Komplexitäten bei Algorithmen

- Komplexitätsangaben können für verschiedene Dimensionen angegeben werden (z.B. Zeit, Speicher, ...)
- wir betrachten Laufzeitkomplexitäten (Dimension: Zeit) in Abhängigkeit von Mengen
- Angabe in der O-Notation -> $O(f(n))$
- Betrachtung von Algorithmen abhängig von Datenkonstellationen
 - Best-Case: Laufzeit im besten Fall
 - Average-Case: Laufzeit im Mittel
 - Worst-Case: Laufzeit im schlechtesten Fall
 - Annahme: Best- und Worst-Case sind Randfälle
- Gängigste Laufzeitkomplexitäten:

O-Notation	Bezeichnung	:-)
$O(1)$	konstant	$O(\text{yeah})$
$O(\log n)$	logarithmisch	$O(\text{nice})$
$O(n)$	linear	$O(k)$
$O(n^2)$	quadratisch	$O(\text{my})$
$O(d^n)$ für $d > 1$	exponential	$O(\text{no})$
$O(n!)$	faktoriell	$O(\text{mg!})$

Arten von inneren Klassen

- Innere Top-Level Klasse
 - Geschachtelte statische Klasse innerhalb einer anderen Klasse mit Bezeichner (Klassenname)
 - können innerhalb und außerhalb (abhängig von der Sichtbarkeit) der Klasse verwendet werden
- Innere Element Klasse
 - Geschachtelte Klasse innerhalb einer anderen Klasse mit Bezeichner (Klassenname)
 - können innerhalb und außerhalb (abhängig von der Sichtbarkeit) der Klasse verwendet werden
 - nur im Kontext eines Objekts der äußeren Klasse
- Innere lokale Klasse
 - Geschachtelte Klasse innerhalb einer Methode mit Bezeichner (Klassenname)
 - können nur innerhalb der Methode (Scope) genutzt werden
- Innere anonyme Klasse
 - geschachtelte Klasse innerhalb einer anderen Klasse / Methode **ohne** Bezeichner
 - werden direkt einer Referenz zugewiesen
 - basieren immer auf einer Klasse (erweitern) oder einem Interface (implementieren)

Innere Top-Level-Klasse

```
package main.inner.toplevelclass;

public class OuterClass {

    // Innerhalb einer anderen Klasse definierte Top-Level Klasse
    public static class InnerTopLevelClass{
        void print(String printText){
            System.out.println(this.getClass().getName() + " " + printText);
        }
    }

    private static void printFromInnerTopLevelClass(String printText) {
        OuterClass.InnerTopLevelClass myInnerTopLevelClass = new OuterClass.InnerTopLevelClass();
        myInnerTopLevelClass.print(printText);
    }

    public static void main(String[] args) {
        OuterClass myClass = new OuterClass();

        System.out.println("OuterClass: " + myClass.getClass().getName());
        OuterClass.printFromInnerTopLevelClass("Inner Top-Level Class: HelloWorld");
    }
}
```

Innere Element-Klasse

```
package main.inner.elementclass;

public class OuterClass {

    // Innerhalb einer anderen Klasse definierte Element Klasse
    public class InnerElementClass {
        void print(String printText){
            System.out.println(this.getClass().getName() + " " + printText);
        }
    }

    void printFromInnerElementClass(String printText){
        OuterClass.InnerElementClass myInnerElementClass = this.new InnerElementClass();

        myInnerElementClass.print(printText);
    }

    public static void main(String[] args) {
        OuterClass myClass = new OuterClass();

        System.out.println("OuterClass: " + myClass.getClass().getName());
        myClass.printFromInnerElementClass("Inner Element Class: HelloWorld");
    }
}
```

Innere lokale Klasse

```
package main.inner.local;

public class OuterClass {

    void printFromLocalInnerClass(String printText){
        // innerhalb einer Methode (Scope) definierte Klasse
        class LocalInnerClass{
            void print(String printText){
                System.out.println(this.getClass().getName() + " " + printText);
            }
        }

        LocalInnerClass myLocalInnerClass = new LocalInnerClass();

        myLocalInnerClass.print(printText);
    }

    public static void main(String[] args) {
        OuterClass myClass = new OuterClass();

        System.out.println("OuterClass: " + myClass.getClass().getName());
        myClass.printFromLocalInnerClass("local inner Class: HelloWorld");
    }
}
```

Innere anonyme Klasse

```
package main.inner.anonym;  
  
public class OuterClass {  
  
    private static interface Printable{  
        void print(String printText);  
    }  
  
    void printFromAnonymousInnerClass(String printText) {  
        // ohne eigenen Bezeichner definiert (kann nicht wiederverwendet werden)  
        // erweitert eine bestehende Klasse oder implementiert ein Interface  
        OuterClass.Printable myAnonymousInnerClass = new OuterClass.Printable() {  
            @Override  
            public void print(String printText) {  
                System.out.println(this.getClass().getName() + " " + printText);  
            }  
        };  
  
        myAnonymousInnerClass.print(printText);  
    }  
  
    public static void main(String[] args) {  
        OuterClass myClass = new OuterClass();  
  
        System.out.println("OuterClass: " + myClass.getClass().getName());  
        myClass.printFromAnonymousInnerClass("Inner anonymous Class: HelloWorld");  
    }  
}
```

Lambda Funktionen (anonyme Funktionen)

- seit Java 8
- reine Funktionen ohne eigene Klasse
- Definition: () -> {}
- implementieren ein funktionales Interface (Interface mit **einer** Methode, **ohne default** Implementierung)
 - ersetzen (unter dieser Voraussetzung) anonyme Klassen
- haben Zugriff auf den umliegenden Kontext (finale / effektiv finale Variablen)
 - in diesem Zusammenhang auch als "Closure" bezeichnet
- verkürzte Schreibweise durch Herleitung der Informationen aus Interface-Definition
- werden an eine Referenz übergeben (direkt oder indirekt)

```
Interface1 lambda1 = parameter -> Anweisung;  
Interface2 lambda2 = (parameter1, parameter2) -> Anweisung;  
Interface3 lambda3 = () -> {  
    Anweisung1;  
    Anweisung2;  
    Anweisung3;  
}
```

Lambda Funktion

```
package main.lambda;  
  
public class OuterClass {  
  
    private static interface Printable{  
        void print(String printText);  
    }  
  
    void printFromLambdaFunction(String printText) {  
        // Lambda Funktionen sind "reine Funktionen" ohne Klasse  
        // nutzen immer ein funktionales Interface (nur eine Methode)  
        // zur Implementierung  
        OuterClass.Printable myLambdaPrintFunction = (lambdaPrintText) -> {  
            System.out.println(this.getClass().getName() + " " + lambdaPrintText);  
        };  
  
        myLambdaPrintFunction.print(printText);  
    }  
  
    public static void main(String[] args) {  
        OuterClass myClass = new OuterClass();  
  
        System.out.println("OuterClass: " + myClass.getClass().getName());  
        myClass.printFromLambdaFunction("Lambda Function: HelloWorld");  
    }  
}
```

Assertion

- Zusicherung / Sicherstellung / Assertion (lat. Aussage / Behauptung)
 - Definition einer Erwartungshaltung zum Vergleich gegen den tatsächlichen Zustand
- JUnit Tests:
 - Klasse: `Assertions` (`org.junit.jupiter.api.Assertions`)
 - statische Methoden zur Definition eines erwartenden Ergebnisses (expected) zum Vergleich mit dem tatsächlichen Ergebnis (actual)
 - überladene Methoden mit zusätzlichem Parameter `Message` für eigene Meldungen
 - automatische Validierung der "Behauptung" durch das JUnit Test-Framework
 - beliebt als statischer Import zur direkten Nutzung der Methoden:

```
import static org.junit.jupiter.api.Assertions.*;
```

- Beispiele:

```
Assertions.assertEquals(<expected>, <actual>[, <Message>]);  
Assertions.assertNotEquals(<expected>, <actual>[, <Message>]);  
Assertions.assertTrue(<actual>[, <Message>]);  
Assertions.assertFalse(<actual>[, <Message>]);  
Assertions.assertTimeout(<expected Duration>, <Executable>[, <Message>]);  
Assertions.assertThrows(<expected Exception-Class>, <Executable>[, <Message>]);
```

JUnit5 - neue Annotationen

Annotation	Beschreibung
<code>@DisplayName("descriptive name")</code>	definiert den Anzeigenamen für den jeweiligen Test / die Testklasse
<code>@Nested</code>	markiert eine innere (geschachtelte) Testklasse -> Strukturierung
<code>@RepeatedTest(count)</code>	sich wiederholender Testfall
<code>@ParameterizedTest</code>	Testfall Parametrisierung -> siehe nächste Slide

@ParameterizedTest

- Separierung von Test-Code und Testfall
- verschiedene Quellen für Testfälle
 - `@ValueSource`
 - `@EmptySource` / `@NullSource` / `@NullAndEmptySource`
 - `@EnumSource`
 - `@CsvSource` / `@CsvFileSource`
 - `@MethodSource`

JUnit5 - Nützliches

Testen von Ausnahmen

```
Exception assertThrows(ExceptionClass, Executable)
```

Testen von mehrer Annotationen auf einmal

```
void assertAll(Executable ...);
```

Testen der Laufzeit

```
void assertTimeout(Duration, Executable);
```

Beispiel: Einfache Test-Klasse

```
import exercises.exkurs.junit.Calculator;
import org.junit.jupiter.api.*;

class CalculatorTest {

    Calculator myCalculator;
    double result = 0;

    @BeforeEach
    void setUp() {
        myCalculator = new Calculator();
        result = 0;
    }

    @Test
    @DisplayName("adding two numbers")
    void add() {
        result = myCalculator.add(5.0, 10.0);
        Assertions.assertEquals(15.0, result);
    }
}
```

F.I.R.S.T. Principal

- **Fast:** Die Testausführung soll schnell sein, damit man sie möglichst oft ausführen kann. Je öfter man die Tests ausführt, desto schneller bemerkt man Fehler und desto einfacher ist es, diese zu beheben.
- **Independent:** Unit-Tests sind unabhängig voneinander, damit man sie in beliebiger Reihenfolge, parallel oder einzeln ausführen kann.
- **Repeatable:** Führt man einen Unit-Test mehrfach aus, muss er immer das gleiche Ergebnis liefern.
- **Self-Validating:** Ein Unit-Test soll entweder fehlschlagen oder gut gehen. Diese Entscheidung muss der Test treffen und als Ergebnis liefern. Es dürfen keine manuellen Prüfungen nötig sein.
- **Timely:** Man soll Unit-Tests vor der Entwicklung des Produktivcodes schreiben.